# Lec 06 : Introduction to PyTorch

## Introduction

In the rapidly evolving field of artificial intelligence, deep learning frameworks serve as indispensable tools for researchers and developers alike. PyTorch stands out as a prominent open-source deep learning framework, empowering users to construct and train sophisticated deep learning models with remarkable flexibility and efficiency. This tutorial provides a foundational understanding of PyTorch, starting from its core components and progressing to the typical pipeline for building and training neural networks.

The journey into PyTorch begins with understanding its fundamental data structure: the tensor. Tensors are the backbone of all operations within PyTorch, enabling everything from simple arithmetic to complex neural network computations. Following this, the lecture delves into Autograd, PyTorch's automatic differentiation engine, which is crucial for efficiently calculating gradients required for model optimization. Finally, these concepts are synthesized through the implementation of a basic neural network, illustrating the practical application of tensors and Autograd in a complete deep learning workflow. As PyTorch is a continuously evolving library, regular consultation of its official documentation is highly recommended to stay updated with its latest features and functionalities.

## Core Concepts

| Term | Definition | Significance |
| --- | --- | --- |
| `Tensor` | A multi-dimensional array, similar to NumPy arrays, that serves as the fundamental data structure in PyTorch. | Enables all data storage and mathematical operations, forming the basis of deep learning models. |
| `Autograd` | PyTorch's automatic differentiation engine that records operations on tensors to compute gradients automatically. | Essential for backpropagation and optimizing neural network parameters during training. |
| `requires_grad` | | |

| Term | Definition | Significance |
| --- | --- | --- |
| | A boolean attribute of a tensor that, when `True`, signals Autograd to track all operations on it for gradient computation. | Determines which tensors contribute to the computational graph and whose gradients will be calculated. |
| **`Leaf Tensor`** | A tensor created directly by the user or one whose `requires_grad` attribute is explicitly set to `True` by the user. | Represents the input nodes in the computational graph, typically model parameters or input data. |
| **`Non-Leaf Tensor`** | A tensor that is the result of an operation on another tensor, especially if any input tensor has `requires_grad=True`. | Intermediate nodes in the computational graph, whose gradients are computed during backpropagation. |
| **`Device`** | Specifies where a tensor is stored and computations are performed, either on the CPU or a GPU (CUDA). | Crucial for leveraging GPU acceleration to speed up computationally intensive deep learning tasks. |
| **`torch.nn.Module`** | The base class for all neural network modules in PyTorch, providing functionalities for defining layers and managing parameters. | Standardizes neural network architecture design, parameter management, and forward pass definition. |
| **`DataLoader`** | An iterator that provides batches of data from a dataset, handling shuffling, | Efficiently manages large datasets by loading data in manageable batches, |

| Term | Definition | Significance |
| --- | --- | --- |
| | batching, and parallel data loading. | preventing memory overload. |
| `optimizer.zero_grad()` | A method called before backpropagation to clear the gradients of all optimized tensors. | Prevents gradient accumulation from previous iterations, ensuring gradients are computed for the current batch only. |
| `optimizer.step()` | A method that performs a single optimization step, updating model parameters based on the computed gradients. | Implements the chosen optimization algorithm (e.g., SGD, Adam) to adjust model weights and biases. |

## Detailed Analysis

### 1. Tensors: The Foundation of PyTorch

Tensors are the core data structure in PyTorch, analogous to NumPy arrays but with added capabilities like GPU acceleration and automatic differentiation. Understanding how to create, manipulate, and inspect tensors is fundamental.

- **Tensor Creation**: Tensors can be created in various ways:
- **From Python lists or numeric values**: Using `torch.tensor()`. A single number creates a scalar (0-dimensional tensor), a list creates a vector (1-dimensional), and nested lists create matrices (2-dimensional) or higher-dimensional tensors.

```
import torch
scalar_tensor = torch.tensor(1)
vector_tensor = torch.tensor([0, 1, 7])
matrix_tensor = torch.tensor([[0, 1, 7], [4, 2, 4]])
```

- **Uninitialized tensors**: `torch.empty(size)` creates a tensor with uninitialized values.

- **Random tensors**: `torch.rand(size)` creates a tensor with random values between 0 and 1 (uniform distribution). `torch.randint(low, high, size)` creates integer tensors within a specified range.
- **Constant tensors**: `torch.zeros(size)` and `torch.ones(size)` create tensors filled with zeros or ones, respectively.
- **From NumPy arrays**: `torch.from_numpy(numpy_array)` creates a tensor that shares memory with the NumPy array. Changes to one reflect in the other.

```
import numpy as np
numpy_array = np.array([1, 2, 3])
tensor_from_numpy = torch.from_numpy(numpy_array)
# Modifying numpy_array will also change tensor_from_numpy
```

- **From NumPy arrays (copy)**: `torch.tensor(numpy_array)` creates a tensor by copying data from the NumPy array, not sharing memory.
- **From other tensors**: Methods like `torch.ones_like(tensor)` or `torch.rand_like(tensor)` create new tensors with the same shape and data type as an existing tensor, but filled with ones or random values.
- **Tensor Attributes and Inspection**:
- **Dimensions**: `tensor.ndim` or `tensor.dim()` returns the number of dimensions. The number of square brackets at one end of the tensor visually indicates its dimension.
- **Shape/Size**: `tensor.shape` or `tensor.size()` returns a `torch.Size` object, which is a tuple indicating the number of elements along each dimension. The number of elements in `torch.Size` equals the tensor's dimension. For example, a `torch.Size([2, 3])` means 2 elements in the 0th dimension and 3 elements in the 1st dimension.
- **Data Type**: `tensor.dtype` returns the data type (e.g., `torch.float32`). `torch.float32` is the default for float values. Data types can be specified during creation ( `dtype=torch.float16` ) or changed later using `tensor.type(torch.double)`.
- **Device Management**: Tensors can reside on either the CPU or a GPU (CUDA).
- **Checking availability**: `torch.cuda.is_available()`
- **Defining device**:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

- **Moving tensors**: `tensor.to(device)` moves a tensor to the specified device.
- **Creating directly on device**: `torch.ones(size, device=device)` creates a tensor directly on the GPU if available, avoiding an extra transfer step.
- **Tensor Indexing and Slicing**: Similar to Python lists and NumPy arrays, tensors support 0-indexed access and slicing.
- `tensor[0]` accesses the first element along the 0th dimension.
- `tensor[1, 0]` accesses the element at index 1 in the 0th dimension and index 0 in the 1st dimension.
- Slicing uses `start:end:step`. `tensor[:, 2]` selects all elements along the 0th dimension but only the element at index 2 in the 1st dimension.
- **Basic Tensor Operations**: Element-wise operations like addition, subtraction, multiplication, and division can be performed using operators (`+`, `-`, `*`, `/`) or PyTorch methods (`torch.add()`, `torch.sub()`, etc.).
- **Tensor Manipulation**:
- **`view()`**: Changes the shape of a tensor without changing its data. The new tensor shares the same underlying memory as the original. `tensor.view(new_shape)`. Using `-1` in `new_shape` allows PyTorch to automatically infer that dimension.
- **`reshape()`**: Can sometimes return a view or a copy, depending on memory contiguity. It's generally safer to use `view()` if memory sharing is desired, or explicitly `clone().detach()` if a copy is needed.
- **`torch.arange(end)`**: Creates a 1D tensor with values from 0 up to (but not including) `end`.
- **`torch.cat(tensors, dim)`**: Concatenates a sequence of tensors along a specified dimension. For example, concatenating two (3,3) tensors along `dim=1` results in a (3,6) tensor.
- **Tensor Reduction Operations**: Operations that reduce the number of dimensions or elements in a tensor.
- **`tensor.sum()`**: Sums all elements in the tensor.
- **`tensor.sum(dim=0)`**: Sums elements along the 0th dimension, reducing that dimension. The resulting tensor will have one less dimension or a reduced size in that dimension.

### 2. Autograd: The Engine for Learning

Autograd is PyTorch's mechanism for automatic differentiation, crucial for training neural networks by computing gradients.

- **Computational Graph**: Autograd internally builds a dynamic computational graph that tracks all operations performed on tensors. This graph allows for efficient calculation of gradients during the backward pass.
- `requires_grad` **Attribute**:
- By default, `requires_grad` is `False` for newly created tensors.
- Setting `requires_grad=True` for a tensor (e.g., `x = torch.tensor(2.0, requires_grad=True)`) tells Autograd to track operations involving this tensor.
- **Leaf vs. Non-Leaf Tensors**: Tensors created by the user with `requires_grad=True` are "leaf" tensors. Tensors that are results of operations on other tensors (where at least one input has `requires_grad=True`) are "non-leaf" tensors. Non-leaf tensors automatically inherit `requires_grad=True` if any of their inputs require gradients.
- `grad_fn` **Attribute**: Non-leaf tensors have a `grad_fn` attribute, which points to the function that created the tensor (e.g., `AddBackward0`, `SigmoidBackward`). This function is called during the backward pass to compute gradients. Leaf tensors do not have a `grad_fn`.
- **Computing Gradients with** `backward()`:
- Calling `output_tensor.backward()` computes the gradients of the `output_tensor` with respect to all leaf tensors that contributed to its computation.
- The computed gradients are stored in the `.grad` attribute of the respective leaf tensors.
- **Gradient Accumulation**: Gradients accumulate by default. If `backward()` is called multiple times without clearing previous gradients, the new gradients will be added to the existing ones.
- **Resetting Gradients**: To prevent accumulation, `tensor.grad.zero_()` should be called before each new backward pass. This is typically done for model parameters in a training loop.

```
# Example of gradient accumulation and resetting
x = torch.tensor(2.0, requires_grad=True)
y = 3 * torch.sigmoid(x) + 5
y.backward()
print(x.grad) # First gradient
```

```
y = 3 * torch.sigmoid(x) + 5 # Recompute y
y.backward()
print(x.grad) # Accumulated gradient (double the first)
x.grad.zero_() # Reset gradient
y = 3 * torch.sigmoid(x) + 5
y.backward()
print(x.grad) # New gradient, not accumulated
```

- **`torch.no_grad()`**: A context manager that disables gradient tracking. This is useful during evaluation or when updating model parameters manually, as it reduces memory consumption and speeds up computations.

```
with torch.no_grad():
# Operations inside this block will not be tracked by Autograd
# No gradients will be computed for tensors created or modified here
```

- **Retaining Gradients for Non-Leaf Tensors**: By default, gradients for intermediate (non-leaf) tensors are released after `backward()` is called to save memory. If you need to inspect the gradient of a non-leaf tensor, you must call `non_leaf_tensor.retain_grad()` before the backward pass.

### 3. Building and Training a Neural Network

The lecture demonstrates building a simple neural network using the MNIST dataset, showcasing the typical PyTorch pipeline.

- **Key Modules**:
- `torch.nn`: Contains all the building blocks for neural networks (layers, activation functions, loss functions).
- `torch.optim`: Provides various optimization algorithms (e.g., SGD, Adam).
- `torchvision.datasets`: Offers pre-built datasets like MNIST.
- `torchvision.transforms`: Provides common image transformations (e.g., converting to tensor).
- **Data Preparation**:
- **Loading Dataset**: MNIST dataset is loaded using `datasets.MNIST()`, specifying `train=True` for training data and `train=False` for test data.
- **Transformations**: `transforms.ToTensor()` converts images to PyTorch tensors.
- **`DataLoader`**: Wraps the dataset to provide iterable batches of data.

```
from torchvision import datasets, transforms
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=tran
train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batch_size=32, shuffl
```

`batch_size` determines the number of samples per batch, and `shuffle=True` randomizes the order of samples in each epoch.
* **Input/Output Dimensions**: For MNIST (28x28 images, 10 classes), the input features are 784 (28*28 flattened), and output features are 10.

- **Neural Network Architecture**:
- A neural network is defined as a Python class inheriting from `nn.Module`.
- The `__init__` method defines the layers (e.g., `nn.Linear` for fully connected layers, `nn.ReLU` for activation).
- The `forward` method defines the forward pass, specifying how input data flows through the layers.

```
import torch.nn as nn
class BasicNeuralNet(nn.Module):
def __init__(self, input_size, hidden_size, num_classes):
super(BasicNeuralNet, self).__init__()
self.fc1 = nn.Linear(input_size, hidden_size)
self.relu = nn.ReLU()
self.fc2 = nn.Linear(hidden_size, num_classes)

def forward(self, x):
out = self.fc1(x)
out = self.relu(out)
out = self.fc2(out)
return out
```

- Model parameters (weights and biases of `nn.Linear` layers) automatically have `requires_grad=True`.
- **Training Loop**:
- **Loss Function**: `nn.CrossEntropyLoss()` is suitable for multi-class classification. It internally applies softmax and then calculates negative log likelihood.
- **Optimizer**: `torch.optim.Adam()` (or other optimizers) is initialized with model parameters and a learning rate.
- **Epoch Loop**: Iterates over the dataset multiple times.
- **Batch Loop**: Iterates through batches provided by the `DataLoader`.

- **Forward Pass**: Input data is passed through the model to get predictions. Images are flattened ( `reshape(-1, input_size)` ) before being fed to the linear layer.
- **Loss Calculation**: The predicted output and ground-truth labels are passed to the loss function.
- **Backward Pass**: `loss.backward()` computes gradients of the loss with respect to model parameters.
- **Optimizer Step**: `optimizer.step()` updates model parameters using the computed gradients.
- **Gradient Zeroing**: `optimizer.zero_grad()` clears gradients before the next iteration to prevent accumulation.

```
# Inside the training loop
optimizer.zero_grad() # Clear gradients
outputs = model(images) # Forward pass
loss = criterion(outputs, labels) # Calculate loss
loss.backward() # Backward pass (compute gradients)
optimizer.step() # Update parameters
```

- **Evaluation**: After training, the model's performance is evaluated on the test set. This involves a forward pass on test data, calculating predictions, and comparing them to true labels to compute accuracy. The `torch.no_grad()` context manager is used during evaluation to disable gradient tracking, saving memory and computation.

### Key Takeaways

- [ ] Tensors are the fundamental data structure in PyTorch, supporting multi-dimensional arrays and operations.
- [ ] PyTorch provides diverse methods for tensor creation, including from data, random initialization, and NumPy arrays.
- [ ] Understanding tensor dimensions, shape, and data types is crucial for effective manipulation.
- [ ] Device management (CPU vs. GPU) is essential for optimizing performance, especially with large models.
- [ ] Autograd automatically computes gradients for tensors with `requires_grad=True`, enabling efficient backpropagation.
- [ ] Gradients accumulate by default; `optimizer.zero_grad()` is necessary to reset them before each optimization step.

- [ ] Neural networks are built as `nn.Module` subclasses, defining layers in `__init__` and the data flow in `forward`.
- [ ] `DataLoader` efficiently manages data by providing batches, crucial for training with large datasets.
- [ ] The typical training loop involves forward pass, loss calculation, backward pass, and optimizer step.
- [ ] `torch.no_grad()` should be used during inference or evaluation to disable gradient tracking and save resources.

## Conclusion

This tutorial has laid the groundwork for understanding PyTorch, from its foundational tensor operations to the sophisticated mechanism of automatic differentiation and the practical steps of building and training a neural network. By mastering tensors, developers gain control over data representation and computation. Autograd, in turn, automates the complex process of gradient calculation, which is the cornerstone of modern deep learning optimization.

The practical example of training a neural network on the MNIST dataset demonstrates how these core concepts integrate into a cohesive workflow. Students are now equipped with the essential knowledge to embark on their own deep learning projects, capable of designing, implementing, and training various models. The flexibility and power of PyTorch, combined with its active community and comprehensive documentation, make it an excellent choice for both academic research and industrial applications in artificial intelligence.